BIASHARA MALL

Phase 0 — Sign up, connect, see my analytics

The thinnest complete loop that delivers real value to a creator. Everything here is trackable: each step ends with something demonstrable. Written 2026-08-16, before any code, so we can tell whether we are on course.

Where we are

STEP		
1	The session layer	✅ 2026-08-16
2	Auth pages	✅ 2026-08-16
3	The dashboard shell	✅ 2026-08-16
4	Connect an account	⬜ next
5	The `Post` model and the time series	⬜
6	Analytics sync	⬜
7	The analytics page	⬜

Steps 1–3 arrived together with the design system — one screen designed properly, then six that inherit it. Detail in `BUILD_LOG.md`.

Deviation from the plan, recorded here so it is not a surprise later: Tailwind was dropped for hand-written CSS with custom properties. The reasoning is in the build log; the short version is that both Tailwind delivery options — a build step, or ~3MB of runtime JS — are the wrong trade for a creator on a cheap Android over patchy data.

The goal, in one sentence

A creator visits the site, signs up, proves they own their TikTok, and sees how their content is actually performing — without anyone running Python.

That is the whole of Phase 0. Not competitors, not hooks, not the store.

Why this slice

It is the smallest thing that is simultaneously:

- **Useful on its own** — "how am I doing?" is a real question people ask daily
- **A full vertical** — proves auth, verification, sync, storage and UI all work together, which no amount of service-layer testing can
- **Testable by a stranger** — you can put it in front of a real creator this week and learn something

Everything after Phase 0 hangs off the same spine. If this works, competitors and hooks are additions, not new systems.

The problem we fix first

We currently throw away the posts analytics needs

`ingestion.py` does this:

```
if not draft.is_product:
    result.skipped_non_products += 1
    return None
```

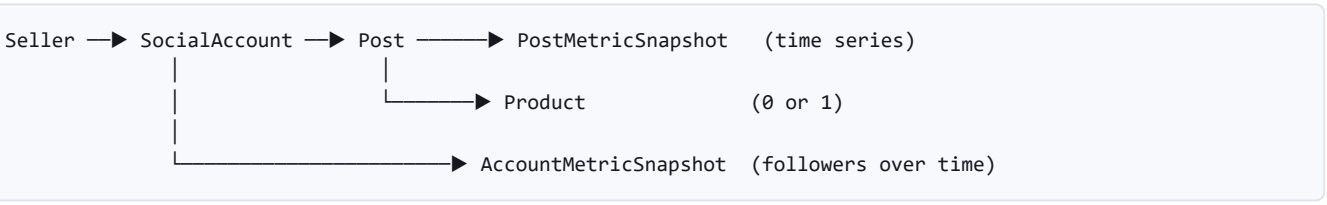
Correct for a store — a talking-head video is not a sellable item, and storing it would leave the seller deleting junk.

Wrong for analytics. A creator's face-to-camera video, their behind-the-scenes clip, their trend participation — those have views, and views are the thing they are paying to understand. We would be showing them a chart with holes in it and calling it analytics.

And `Product` is the wrong home for content

Today `Product` is *a post plus commerce fields*. A creator who signs up for content tools and never sells anything would have their videos stored as "products", and every storefront query would need to remember to filter them out.

The split:



- `Post` is content. *Every* post, product or not. Carries the caption, cover, hashtags and the latest metrics.
- `Product` is commerce. Points at the Post it came from — or at nothing, for a manual photo upload.
- **Snapshots** are history. Neither Post nor Product can answer "am I growing?" on their own.

One thing that gets cheaper

Analytics ingestion **does not call the AI at all.** Scrape, store, done. No Gemini, no cascade. Only commerce ingestion pays for drafting.

That matters at $10/month: the feature everyone uses daily is the cheap one.

What gets built, in order

Each step is demonstrable. Do not start the next until the previous one is real.

Step 1 — The session layer ✅

Nothing renders for a logged-in user until this exists.

- `current_account` dependency — reads the session cookie, loads the account
- `require_login` — redirects to `/login` rather than returning a bare 401, because this is a browser, not an API
- Cookie set on login, cleared on logout: `HttpOnly`, `SameSite=Lax`, `Secure` in production only (so localhost works)

Demonstrable: a route that says who you are, or bounces you to login.

Step 2 — Auth pages ✅

- `GET/POST /signup` — email, password, shop or creator name
- `GET/POST /login`
- `POST /logout`
- Base layout, the design system, error states that keep the typed values

The services exist and are tested. This is the HTTP and HTML around them.

Demonstrable: sign up in a browser, land logged in, log out, log back in.

Step 3 — The dashboard shell ✅

- `/dashboard` — navigation, account menu, and an honest empty state
- The empty state is the product's first impression: "Connect your TikTok to see how your content is doing", one button, nothing else

Demonstrable: a logged-in creator sees a real page telling them what to do next.

Step 4 — Connect an account

Wires the verification service — already built and tested — to real screens.

- Enter handle → claim created → **show the `soko-XXXXXX` code** with copy button
- Clear instructions: paste into your TikTok bio, save, come back
- "I've added it" → check → connected, or a plain explanation of why not
- Attempts and expiry surfaced honestly, because both already exist in the model

Demonstrable: connect a real TikTok account end to end, in a browser.

> Rename note: the code prefix is still `soko-`. Changing it to `bm-` is a one-line change but invalidates any code already in someone's bio. Nobody has one yet, so now is the moment — decide in Step 4.

Step 5 — The Post model and the time series

The migration. No UI.

- `Post` — platform, platform_post_id, caption, hashtags, cover, posted_at, and the *latest* metrics denormalised for fast listing
- `PostMetricSnapshot` — post_id, captured_at, views, likes, comments, shares
- `AccountMetricSnapshot` — social_account_id, captured_at, followers, total likes
- `Product.post_id` — nullable, because manual uploads have no post
- Move platform/caption/metrics fields off `Product` onto `Post`

> **Start writing snapshots the moment this ships.** History cannot be backfilled. Every day without it is a day of data gone permanently — this is the only thing in the plan that is more expensive to delay than to do.

Demonstrable: migration applies, tests green, a sync writes rows to all three tables.

Step 6 — Analytics sync

- `sync_posts()` — scrape → upsert Posts → append snapshots. **No AI.**
- Keeps the existing once-per-day cooldown; it is the same scrape either way
- Stores *every* post, product or not

Demonstrable: connect an account, hit sync, see rows appear.

Step 7 — The analytics page

- Header: followers, total views, posts — with change since the last snapshot
- Trend: followers and views over time (a real chart, not a number)
- Post table: sortable by views, comments, engagement rate, date
- **Your average** — the baseline everything else is measured against, and the number competitor benchmarking will need in Phase 2
- Honest empty states: "one sync so far — check back tomorrow for a trend"

Demonstrable: the loop is complete. A stranger can sign up and learn something true about their content.

What Phase 0 deliberately does not include

	WHY
Competitor search	Phase 2. Needs the creator's own baseline first, which is Step 7
Hooks and scripts	Phase 3. Needs the corpus, which needs competitors
Canva / CapCut	Phase 3.5. Canva is on a waitlist anyway
Catalogue dashboard	Commerce. The storefront already works; nobody is asking for it yet
WhatsApp	Phase 7. Nothing here depends on it

Open decisions

1. **Verification code prefix** — `soko-` or `bm-`? One-line change, and now is the only free moment. Step 4.
2. **What happens to the seeded demo shop?** It has `Product` rows built on the old shape. Simplest is to re-seed after the migration; the data is regenerable by design.
3. **Chart rendering** — server-rendered SVG keeps the page fast on a cheap phone and adds no dependency. A JS chart library is prettier and heavier. Decide at Step 7.

How we will know it worked

Not "the code is written". The test is:

> Hand the URL to a Kenyan creator who has never seen it. They sign up, connect their TikTok, and tell you something they learned about their own content.

If they get stuck, that is the finding — and it is worth more than the next feature.
